# Applying Engineering Principles in InstiChain

Faculty-authorized campus records on Hyperledger Fabric

11 October 2026

## Project Lifecycle and Engineering Strategy

The project is developed in small, reviewable stages:

- **Foundation:** React frontend, FastAPI backend, configuration templates, and PostgreSQL migrations.
- **Identity:** institute-email verification, single-use links, persistent sessions, and controlled administrator bootstrap.
- **Institution setup:** club creation, advisor onboarding, and club-scoped authorization.
- **Blockchain transition:** Hyperledger Fabric replaced the local hash chain as the authoritative institutional ledger.
- **Domain workflows:** signed PoR grants/revocations, competitions, immutable results, and advisor-signed achievements.

The architecture has clear ownership rules:

- **Fabric:** authoritative institutional events and their ordered audit history.
- **PostgreSQL:** users, sessions, private operational data, and fast query projections.
- **SMTP/Google Drive:** email delivery and evidence files outside the blockchain.
- **FastAPI:** authorization, validation, and coordination between these systems.

A PostgreSQL projection changes only after Fabric confirms the transaction. Expiring tokens, evidence bytes, OAuth secrets, and private keys are never placed on the shared ledger.

**Lifecycle rule:** decompose by behavior and trust boundary, implement the smallest complete slice, verify normal and failure paths, document the operational change, then commit it independently.

## Prompt Engineering

Prompts are treated as engineering specifications, not one-line requests. The following principles emerged through the project:

- **Give context:** name the stack, current feature, actor, and architecture boundaries.
- **Ask for one outcome:** keep each prompt small enough for one focused commit.
- **Define behavior:** state inputs, outputs, permissions, and data changes.
- **Include failure cases:** cover invalid signatures, wrong roles, expired links, duplicates, and unavailable services.
- **Protect existing work:** inspect first, preserve unrelated edits, and stage only intended files.
- **Demand evidence:** run tests and report changed files, limitations, and the proposed commit message.

This changed a broad request such as “replace the backend with blockchain” into a verifiable sequence: create canonical event bytes, sign locally, submit to Fabric, wait for confirmation, and only then update the PostgreSQL projection. If Fabric fails, the local domain change is rolled back.

## Requirements Traceability

| Class principle | Project application | Evidence |
|---|---|---|
| **Incremental design** | Dependency-ordered vertical slices | Git history and staged milestones |
| **Security by design** | Scoped roles, local Ed25519 signing, private/off-chain secrets | Architecture and auth services |
| **Failure atomicity** | Projection follows confirmed Fabric write | Authority/service tests |
| **Traceability** | Requirement, test, docs, and commit align | Conventional feature commits |

## Structured Debugging

Debugging follows a repeatable evidence-driven loop:

1. **Reproduce and classify.** Record the exact command, request, response code, logs, environment, and expected behavior. Decide whether the fault is browser, API, database, SMTP, Docker, Fabric gateway, or chaincode.
2. **Check dependencies from the outside in.** Use health/status commands, container state, port ownership, configuration presence, migrations, and then service logs. A `401` from `/api/auth/session` while logged out is expected; a failed health check is not.
3. **Form one falsifiable hypothesis.** Change one variable and use the smallest direct test: API call before UI, Mailpit inbox before real SMTP, Fabric query before PostgreSQL projection, or a focused unit test before the full suite.
4. **Fix the root cause and add a guard.** Prefer explicit startup errors, bounded state transitions, and reusable status commands over hiding symptoms.
5. **Verify and document.** Re-run the reproducer and regression tests, inspect the diff, and update setup and troubleshooting instructions when an operational assumption changed.

Examples from development:

- **Docker socket error:** checked the host service manager first, removed the assumption that `systemd` always exists, and added Docker/database/Mailpit status targets.
- **Email accepted but not received:** separated API acceptance from delivery and confirmed that Mailpit captures local mail at port 8025 instead of sending it externally.
- **Login service unavailable:** checked database health, migrations, SMTP settings, and backend logs before changing the frontend.
- **Occupied development ports:** stopped only Uvicorn/Vite processes belonging to this checkout rather than killing unrelated processes.

## Version Control (Git)

Git is used as an audit trail for both implementation and decisions:

- Each commit represents one capability, for example advisor access, PoR grant, PoR revocation, competitions, immutable results, or credential issuance.
- Recent commits use clear prefixes such as `feat`, `fix`, and `test` so their purpose is easy to scan.
- Before committing, `git status -short` finds unrelated changes and `git diff -cached` verifies the exact staged patch.
- Targeted `git add` commands prevent unrelated work from entering a feature commit.
- Prompts and decisions are summarized in commit messages and project documentation when they affect the design.
- `.env`, keys, virtual environments, dependencies, build output, and Fabric runtime identities are ignored.

## Testing, Refactoring, and Documentation

### Testing

- Backend pytest tests cover migrations, email verification, sessions, administrator operations, and service preflight checks.
- Node tests cover Fabric chaincode behavior.
- Negative tests cover wrong domains/roles/clubs, expired links, invalid signatures, duplicate requests, immutable versions, and Fabric failure rollback.
- Frontend checks currently include TypeScript validation and a production Vite build. Browser/component tests remain future work.

### Refactoring

- The old PostgreSQL hash chain remains read-only as `LegacyLedgerEntry`; new institutional writes use Fabric.
- Shared session checks, canonical event construction, and projection rules reduce duplicated security logic.
- External behavior is preserved and regression tests are run after structural changes.

### Documentation

- The README contains setup and demo commands; the architecture document explains component and data ownership.
- Schema, administrator bootstrap, and Fabric operations have separate focused guides.
- Safe `.env.example` files document configuration without exposing secrets.
- Make targets turn start, stop, and status instructions into repeatable commands.

## Appendix: Sample Prompts

The examples below show how broad conversational requests were converted into precise, testable prompts. They are shortened to highlight the engineering principles used.

### A. Architecture migration

#### Initial request

> "I would like a blockchain in the backend. Hyperledger would be my choice. Please replace the current backend with that."

#### Engineered prompt

> Inspect the existing FastAPI/PostgreSQL implementation first. Keep FastAPI as a thin API. Make Hyperledger Fabric authoritative only for institutional events; retain PostgreSQL for users, sessions, private data, and query projections. Submit a signed canonical event to Fabric and update the projection only after commit confirmation. Keep email, evidence bytes, OAuth tokens, and private keys off-chain. Preserve unrelated edits, add failure/rollback tests, run them, and provide targeted staging and commit commands without committing automatically.

**Why it is better:** it defines the architecture, data ownership, transaction order, security exclusions, failure behavior, verification, and change-control boundary.

### B. Credential revocation

#### Initial request

> "Advisor can revoke PoRs."

#### Engineered prompt

> Add prepare/submit endpoints for advisor-signed PoR revocation. The prepare step returns deterministic canonical bytes; submit verifies the Ed25519 signature, active advisor appointment, club scope, request expiry, and idempotency key. Record an immutable revocation event on Fabric, retain the original grant, and update PostgreSQL only after ledger confirmation. Test wrong-club access, invalid/expired signatures, duplicate submission, already-revoked roles, concurrent handover, and Fabric rollback.

**Why it is better:** the prompt converts an ambiguous delete operation into an auditable state transition with explicit authorization and race conditions.

### C. Debugging an email failure

#### Observed symptom

> "The backend returned 202 Accepted, but I did not get an email" and later "service not available."

#### Engineered diagnostic prompt

> Reproduce the request and capture browser network details plus backend logs. Check `/api/health`, PostgreSQL container health, Alembic revision, SMTP host/port, and Mailpit status/inbox in that order. Confirm whether the environment is local capture or real delivery. Call the email endpoint directly to isolate the UI. Do not weaken the generic 202 anti-enumeration response. Fix the failing boundary, add a regression/preflight test, and document the exact startup/status command.

**Why it is better:** it distinguishes successful request acceptance from downstream delivery, preserves a security property, and produces a reproducible diagnostic path.

### D. Standard template for future commits

> **Context** (inspect current code and invariants) → **one outcome** (actor and behavior) → **constraints** (authorization, privacy, atomicity, idempotency) → **failure cases** → **tests and docs** → **reviewable Git commands**.

This template makes AI assistance accountable: generated code is not accepted because it appears plausible; it must fit the architecture, survive negative tests, preserve unrelated work, and leave a readable history.